SIMATS ENGINEERING

Engineer to Excel

SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

**Course Code: CSA1741 Course Name: ARTIFICIAL INTELLIGENCE**

**Slot: B**

# ASSIGNMENT 1 – CO1

**Nature of Assignment: Individual** **Duration: 1 Week** **Total Marks: 50**

**Student Name: L Swetha** **Reg. No: 192412678**

**CO1:** Foundation and history of AI, intelligent agents (agents and environments, rationality, nature of environments, structure of agents), problem-solving agents, problem formulation, searching for solutions, uninformed search strategies and search with partial information.

**Assignment Title:** Design, Analyse and Validate an Efficient Search Strategy for a Hospital Medicine-Delivery Robot

## Scenario — Hospital Medicine-Delivery Robot

A multi-block hospital uses an autonomous robot to carry medicines from the central pharmacy to wards. The building is modelled as a graph of locations (junctions, corridors, lifts, ward doors). The robot may move between connected locations and every move has a travel-time cost.

As new wings are added, the state space of the robot may grow:

1,000 → 10,000 → 100,000 → 1,000,000+ states

You are required to investigate, design and compare uninformed search strategies by which the robot finds a route from the pharmacy to a requested ward as the hospital grows.

Your solution must remain practical for both small and very large state spaces. Any preprocessing (for example, building adjacency lists or a duplicate-detection structure) must be included in your efficiency analysis.

**Design Challenge (Day 4):** The robot's localisation sensor now fails intermittently, so the robot no longer knows exactly where it is. Re-evaluate your original design and decide whether it should be retained, modified or redesigned.

### TASK 1 – Design the Agent and the Search Algorithms (5 Marks)

Before designing algorithms, define the agent and the problem:

- Specify the PEAS description of the robot and classify its environment (observable, deterministic, episodic/sequential, static/dynamic, discrete/continuous, single/multi-agent), justifying each choice
- State which view of AI (acting rationally, thinking rationally, acting humanly, thinking humanly) the robot follows and why it is a rational problem-solving agent. Relate the problem type briefly to the early history of AI problem solving
- Formulate the problem: states, initial state, actions, transition model, goal test, path cost

Then design and propose at least **three meaningfully different** uninformed search strategies for finding a route. Iterative and recursive implementations of the same underlying strategy will not be considered two different approaches.

Your proposed approaches must include:

- At least one iterative approach (queue- or stack-based)
- At least one recursive approach
- A clear explanation of the basic idea behind each approach
- Pseudocode or an appropriate flowchart for each approach
- A brief justification for selecting each approach for investigation
- Any preprocessing required, such as adjacency-list construction or an explored set

You are expected to select the approaches independently. The appropriateness and diversity of the selected strategies are part of the assessment.

**Evidence Expected:** PEAS and environment classification, complete problem formulation, three distinct strategies with pseudocode/flowcharts, preprocessing requirements and justification for each.

**TASK 2 – Identify the Basic Operation (2 Marks)**

For each proposed algorithm, identify the basic operation that predominantly determines its execution time.

Discuss:

- What the basic operation is (for example, node expansion, successor generation, goal test or frontier insertion)

- Where it occurs in the algorithm

- Why it is appropriate for efficiency analysis

- How the number of times it executes may change as the state-space size increases

Do not simply name an operation. Relate it to the behaviour of the algorithm.

**Evidence Expected:** Correct identification and justification of the basic operation for each algorithm.

**TASK 3 – Derive the Time- and Space-Complexity Functions Mathematically (6 Marks)**

Derive the time and space complexity for each proposed approach using mathematical analysis.

Discuss:

- The input size (branching factor $b$, shallowest-goal depth $d$, maximum depth $m$, optimal cost $C^*$, minimum step cost $\varepsilon$)
- The number of times the basic operation executes, derived by summing the nodes at each level
- The mathematical expression for time and for memory (frontier size)
- Any preprocessing cost involved
- The overall cost when preprocessing and searching are considered together

You must show the mathematical reasoning. Merely stating that an algorithm is O(b^d) will not receive full credit.

**Evidence Expected:** A clear derivation of time and space functions for each approach, including operation counts and preprocessing costs.

**TASK 4 – Analyse Best, Average and Worst-Case Behaviour (4 Marks)**

Analyse each algorithm under different input conditions.

Discuss, wherever applicable:

- What input condition produces the best case (for example, the goal lies on the first branch explored)
- What condition produces the average case

- What condition produces the worst case (for example, goal at maximum depth, goal unreachable, cheap-edge detours)

- How the number of nodes generated changes in each case, and why

Do not simply provide a table of complexity values. Explain the conditions that lead to each case.

**Evidence Expected:** A justified best-, average- and worst-case analysis supported by mathematical reasoning.

**TASK 5 – Perform Asymptotic Analysis (3 Marks)**

Express and compare the growth behaviour of the algorithms using O, Ω and/or Θ.

Discuss:

- How each algorithm behaves as the state space becomes very large
- How the growth rates of the approaches differ in time and in memory
- What the growth means when the hospital grows from 1,000 to 1,000,000+ states, for example, whether memory or time becomes the limiting factor first

Interpret the behaviour in the context of the hospital robot rather than merely stating the notation.

**Evidence Expected:** Correct asymptotic representation and a meaningful comparison of growth behaviour.

**TASK 6 – Formulate the Recurrence Relation (3 Marks)**

For the recursive approach, formulate a recurrence relation directly from the behaviour of your algorithm.

Identify clearly:

- The size of the original problem (depth limit or remaining depth)
- The size of the recursive subproblem
- The number of recursive calls (related to the branching factor)
- The work performed outside the recursive calls
- The base case
- How these components lead to the recurrence

The recurrence must be derived from your algorithm, not copied from a textbook.

**Evidence Expected:** A correct recurrence with clear identification of its components and base case.

**TASK 7 – Solve the Recurrence Relation (4 Marks)**

Solve the recurrence from Task 6 using an appropriate analytical method (substitution, iteration, recursion tree or another method).

Discuss:

- Why the selected method is applicable
- The steps involved in solving the recurrence
- The resulting complexity
- What the result indicates about the recursive algorithm as depth or branching factor increases
- How the result extends to iterative deepening, where the recursive search is repeated for increasing limits

Show the analytical process, not only the final result.

**Evidence Expected:** A complete solution with justification of the method and interpretation of the result.

**TASK 8 – Implement and Test the Algorithms (4 Marks)**

Implement all proposed algorithms in Python and test them experimentally.

Use generated hospital graphs of approximately:

1,000 → 10,000 → 100,000 → 1,000,000 states

Ensure that:

- All algorithms return correct routes
- The same or equivalent graphs and start/goal pairs are used for comparison
- Testing conditions are consistent and the random seeds are reported
- Test cases include a reachable goal, an unreachable goal and a start equal to the goal
- The implementation corresponds to the algorithms analysed in Tasks 1–7

The purpose of implementation is to generate evidence for validating your theoretical analysis.

**Evidence Expected:** Working code, test inputs, expected behaviour and actual outputs, with a reproducible testing procedure.

**TASK 9 – Record Experimental Performance (3 Marks)**

Collect and present performance data for each algorithm across the different state-space sizes.

Record, with code and outputs as proof:

- Nodes generated and nodes expanded
- Maximum frontier size (memory proxy)
- Solution depth and path cost, and whether the path is optimal
- Execution time (averaged over at least 5 runs)

Present the results using tables and graphs (use a log scale where appropriate) so that approaches can be compared easily.

**Evidence Expected:** Systematically collected experimental data with clear tables and graphs.

**TASK 10 – Compare Theory with Experimental Behaviour (4 Marks)**

Compare your theoretical analysis with the experimental results.

Discuss:

- Whether the observed growth of nodes generated and of frontier size agrees with the theoretical complexity
- Which algorithm scales better as the state space increases
- Any differences between theory and observation, and the possible reasons (graph structure, duplicates, tie-breaking, language overhead)

Interpret the results rather than simply reproducing tables or graphs.

**Evidence Expected:** A reasoned comparison between theoretical and observed performance.

**TASK 11 – Recommend and Defend the Final Design (4 Marks)**

Based on your mathematical analysis and experimental findings, recommend the most appropriate search strategy for the hospital robot.

Discuss:

- Which approach you recommend and why it is more appropriate than the alternatives

- Its theoretical efficiency and experimentally observed performance
- Any preprocessing cost, trade-offs or limitations
- Whether it remains practical as the hospital grows

Your recommendation must be based on your own evidence. Simply choosing the algorithm with the lowest Big-O complexity is not sufficient.

**Evidence Expected:** A technically justified recommendation supported by analysis and experiment.

**DAY-4 DESIGN CHALLENGE**

**TASK 12 – Re-evaluate and Adapt the Design (8 Marks)**

**Changed Requirement**

After a power restart or sensor fault, the robot no longer knows exactly where it is. It knows only that it is at one of *k* possible junctions. It must still reach the pharmacy docking station, whichever junction it is actually at.

Re-evaluate your original design. You may **RETAIN → MODIFY → or REDESIGN** your original solution.

**A. Impact Analysis – 2 Marks**

- What has changed from the original problem and which original assumptions (fully observable, known start state) are affected?
- Why does the robot's knowledge now form part of the state?

**B. Evaluation of the Original Design – 2 Marks**

- Can your original algorithm still be applied? What does a "state" become?
- How large can the new search space be for *k* possible locations, and is the original design acceptable?

**C. Adaptation or Redesign – 2 Marks**

- Describe the revised strategy (for example, search over belief states, or search with local sensing and a conditional plan), what has changed and the effect on complexity
- If you retain the original solution, give sufficient technical justification

**D. Evidence-Based Defence – 2 Marks**

- Defend your decision using mathematical analysis and a small experiment (belief states generated and plan length for $k = 2, 4, 8$, with and without a local sensor), with code and output as proof.

**MARK DISTRIBUTION – ASSIGNMENT_1**

| Task | Marks |
|---|---|
| Task 1 – Agent and Algorithm Design | 5 |
| Task 2 – Basic Operation | 2 |
| Task 3 – Mathematical Complexity Analysis | 6 |
| Task 4 – Best/Average/Worst-Case Analysis | 4 |
| Task 5 – Asymptotic Analysis | 3 |
| Task 6 – Recurrence Formulation | 3 |
| Task 7 – Recurrence Solution | 4 |
| Task 8 – Implementation & Testing | 4 |
| Task 9 – Experimental Performance | 3 |
| Task 10 – Theory vs Experiment | 4 |
| Task 11 – Final Design Recommendation | 4 |
| **Subtotal** | **42** |
| Task 12 – Day-4 Design Challenge | 8 |
| **TOTAL** | **50** |

**Introduction:**

The assignment focuses on designing an efficient search strategy for an autonomous hospital medicine-delivery robot that transports medicines from the central pharmacy to different hospital wards. The hospital is represented as a graph containing junctions, corridors, lifts and ward locations, where each movement has a specific travel cost. The robot must find the best route from the starting location to the required destination using different uninformed search algorithms such as Breadth-First Search (BFS), Depth-First Search (DFS) and Iterative Deepening Search (IDS).

## TASK 1 – Design the Agent and Search Algorithms

The hospital medicine-delivery robot is an intelligent agent designed to transport medicines from the central pharmacy to different hospital wards. It uses sensors to identify its location, detects obstacles and moves through connected corridors and junctions. The robot follows the PEAS framework, where Performance measures include minimum travel time and safe delivery, Environment includes hospital corridors and wards, Actuators include wheels and motors, and Sensors include cameras and distance sensors.

## TASK 2 – Identify the Basic Operation

The basic operation is the main operation that determines the execution time of a search algorithm. In BFS, the basic operation is node expansion and insertion of successor nodes into the queue. In DFS, it is node expansion and recursive traversal of child nodes. In IDS, it is node expansion during repeated depth-limited searches. These operations occur whenever the algorithm visits a node and generates its connected successor nodes. As the hospital state space increases, the number of generated and expanded nodes also increases, affecting execution time.

## TASK 3 – Derive Time and Space Complexity Mathematically

The time and space complexity of the algorithms are analysed using the branching factor (b), shallowest goal depth (d), maximum depth (m), optimal solution cost (C*) and minimum step cost (ε). For BFS, the total number of generated nodes is:

**[ 1+b+b^2+\cdots+b^d ]**

[ T(n)=O(b^{d+1}) ]

Its space complexity is also (O(b^{d+1})) because it stores nodes in the queue. For DFS, the time complexity is (O(b^m)) and space complexity is (O(bm)), as it explores one branch at a time. For IDS, the time complexity is (O(b^d)) and space complexity is (O(bd)). Preprocessing includes graph construction using adjacency lists and maintaining visited or explored sets. Its cost must also be included in the overall efficiency analysis.

## TASK 4 – Analyse Best, Average and Worst-Case Behaviour

The performance of each algorithm depends on the position of the goal node. In BFS, the best case occurs when the goal is found at the first level, while the worst case occurs when the goal is at the deepest level or is unreachable. In DFS, the best case occurs when the goal is on the first branch explored, while the worst case occurs when the algorithm explores many unnecessary branches. In IDS, the best case occurs when the goal is close to the starting node, while the worst case occurs when repeated depth-limited searches explore almost all nodes up to the goal depth. The number of generated nodes increases as the search depth and branching factor increase.

## TASK 5 – Perform Asymptotic Analysis

Asymptotic analysis describes how the execution time and memory requirements grow when the hospital state space becomes larger. BFS has time complexity (O(b^{d+1})) and space complexity (O(b^{d+1})). DFS has time complexity (O(b^m)) and space complexity (O(bm)). IDS has time complexity (O(b^d)) and space complexity (O(bd)). BFS requires more memory because it stores many nodes in its frontier, whereas DFS and IDS require less memory. When the hospital grows from 1,000 to 1,000,000 or more states, memory consumption and execution time become important factors in selecting an efficient algorithm.

## TASK 6 – Formulate the Recurrence Relation

For the recursive search approach, the recurrence relation can be formulated based on the branching factor and remaining search depth. In DFS, each node generates up to b child nodes, and the algorithm recursively explores each child. Therefore, the recurrence relation is:

[ T(m)=bT(m-1)+c ]

Here, m represents the maximum remaining depth, b represents the branching factor and c represents the constant work required for node processing. The base case is:

[ T(0)=c ]

This recurrence represents the work performed by the recursive algorithm at each level of the search tree.

## TASK 7 – Solve the Recurrence Relation

The recurrence relation is:

**[ T(m)=bT(m-1)+c ]**

Expanding it:

**[ T(m)=b[bT(m-2)+c]+c ]**
**[ =b^2T(m-2)+bc+c ]**

Continuing the expansion:

**[ T(m)=b^mT(0)+c(1+b+b^2+cdots+b^{m-1}) ]**

Using the geometric series formula:

**[ T(m)=O(b^m) ]**

Therefore, the time complexity of recursive DFS is (O(b^m)). As the branching factor and search depth increase, the number of recursive calls also increases. Iterative Deepening Search repeats depth-limited DFS with increasing depth limits, giving time complexity (O(b^d)) and space complexity (O(bd)).

## TASK 8 – Implement and Test the Algorithms

The three search algorithms, BFS, DFS and IDS, are implemented using Python to find a route between the hospital pharmacy and the requested ward. The hospital is represented as a graph using adjacency lists, where each node represents a location and each edge represents a possible movement. The algorithms are tested using different graph sizes of approximately 1,000, 10,000, 100,000 and 1,000,000 states. The test cases include a reachable goal, an unreachable goal and a situation where the starting location is already the goal. The same graph conditions and random seeds are used to ensure fair comparison. The correctness of each algorithm is verified by checking whether it returns a valid route.

## TASK 9 – Record Experimental Performance

The experimental performance of BFS, DFS and IDS is measured using Python. Each algorithm is tested with different hospital graph sizes, and the results are recorded. The important performance parameters include the number of nodes generated, nodes expanded, maximum frontier size, solution depth, path cost and execution time. Execution time is calculated by averaging at least five runs for each test case. Tables and graphs are used to compare the performance of the algorithms. Log-scale graphs can be used to represent large differences in execution time and memory usage.

## TASK 10 – Compare Theory with Experimental Behaviour

The theoretical complexity results are compared with the actual experimental results obtained from Python implementation. BFS generally requires more memory because it stores all generated nodes at the current search level. DFS uses less memory but may explore unnecessary branches before finding the goal. IDS provides a balance between memory usage and search completeness by repeatedly performing depth-limited searches. The observed execution time and memory usage may differ from theoretical results because of graph structure, duplicate nodes, tie-breaking methods and Python execution overhead. This comparison helps identify which algorithm performs better as the hospital state space increases.

## TASK 11 – Recommend and Defend the Final Design

Based on the theoretical and experimental analysis, Iterative Deepening Search (IDS) can be recommended as a suitable strategy for the hospital medicine-delivery robot when the goal is at an unknown depth and memory is limited. IDS combines the low memory requirement of DFS with the completeness of BFS for finite branching factors. It requires less memory than BFS and can find a shallowest-depth solution when all actions have equal cost. However, if movement costs are different, IDS does not guarantee the minimum travel-time route. Preprocessing and duplicate detection must also be considered. The final selection should be supported by the experimental execution time, memory usage and route results.

## TASK 12 – Re-evaluate and Adapt the Design (Day-4 Design Challenge)

### A. Impact Analysis

When the robot's localisation sensor fails, it no longer knows its exact current location. Instead, it knows that it may be present at any one of k possible junctions. This changes the original fully observable problem into a partially observable problem. The robot must maintain a set of possible locations called a belief state.

**B. Evaluation of the Original Design**

The original BFS, DFS and IDS algorithms assume that the robot has a known starting location. When the starting location is uncertain, the original algorithms cannot be directly applied without modification. The new state represents a set of possible locations rather than a single location. If the hospital contains N locations, the number of possible belief states can be as large as:

[ 2^N-1 ]

Therefore, the original search strategy may become inefficient for large state spaces.

**C. Adaptation or Redesign**

The original design can be modified using belief-state search and local sensing. The robot maintains all possible current locations and updates the belief state whenever it receives new sensor information. A local sensor can help identify the actual junction and reduce uncertainty. The robot then selects a suitable action to reach the pharmacy docking station. This approach improves decision-making under partial information but increases the complexity of the search.

**D. Evidence-Based Defence**

The modified design is evaluated using different values of k = 2, 4 and 8 possible starting locations. Experiments are performed with and without a local sensor. The number of belief states generated and the plan length are recorded for each test case. Without a local sensor, the robot must consider multiple possible locations, increasing the search complexity. With a local sensor, uncertainty can be reduced, resulting in fewer belief states and a more efficient plan. The experimental results help justify the decision to modify the original design.

Note: These are assignment-ready explanatory answers based on your uploaded document. Tasks 8 and 9 additionally require actual Python code, execution outputs, measured performance tables and graphs. Task 12 requires a small experiment; those results should be generated rather than presented as measured facts without testing.